Universidad Distrital Francisco José de Caldas

School of Engineering

Computer Engineering

Workshop 3 — Reinforcement Learning

Real Estate Investment Decision-Making

Course: Machine Learning — Semester 2026-III
Team: Group 1

Authors

Chavez Porras, Miguel Alejandro
Guerrero Ordeñez, Luisa Fernanda
Arboleda Zambrano, Dilan Stive

2026

Contents

1 Introduction **2**

2 MDP Formulation **2**

2.1 State Space (S) 2

2.2 Action Set (A) 2

2.3 Transition Dynamics (P) 3

2.4 Reward Function (R) 3

2.5 Episode Termination 3

3 Environment Implementation and Validation **3**

3.1 Random Policy Validation 4

4 Agent Algorithm (DQN) **4**

4.1 Architecture 4

4.2 Training Dynamics 5

5 Hyperparameter Search **6**

5.1 Results Table 6

5.2 Analysis 6

6 Evaluation and Comparison **7**

6.1 Final Test Metrics 7

6.2 Comparison with Workshops 1 and 2 7

7 Trajectory Visualizations **7**

8 Discussion **8**

9 References **9**

1. Introduction

Our project domain is **real estate valuation**. In Workshops 1 and 2, we developed supervised regression models to predict house prices using the Ames Housing dataset. However, price prediction is a *static* task: it answers "how much is this house worth?" but not "what should an investor do with it over time?"

In this workshop, we reformulate the problem as a **sequential decision-making task** using Reinforcement Learning (RL). An agent (the investor) must decide month by month whether to `WAIT`, `BUY`, `RENOVATE`, or `SELL` a property to maximize cumulative net profit. RL is appropriate here because decisions are sequential, rewards are delayed, and the environment is stochastic.

2. MDP Formulation

This section summarizes the document `mdp_formulation.pdf`, previously submitted. We model the investment task as a finite-horizon Markov Decision Process defined by the tuple

$$(S, A, P, R, \gamma, T).$$

2.1 State Space (S)

The state is an 8-dimensional continuous vector containing:

1. Current property price (normalized).
2. Market average price.
3. Price trend (last 3 months).
4. Ownership flag (0 or 1).
5. Property condition (0 to 1).
6. Available capital (normalized).
7. Months held.
8. Time step (t/T).

2.2 Action Set (A)

The action space is discrete with four actions: `0` (`WAIT`), `1` (`BUY`), `2` (`RENOVATE`), and `3` (`SELL`). Invalid actions are handled via a penalty.

2.3 Transition Dynamics (P)

The property price follows a geometric random walk with drift:

$$P_{t+1} = P_t \exp(\mu + \sigma z),$$

where z represents a standard normal random variable. The property condition decays over time and increases upon renovation. Capital is updated through buying, selling, and maintenance costs.

2.4 Reward Function (R)

We use a dense + terminal reward scheme:

- **SELL:** Profit = sale price − purchase price − costs.
- **RENOVATE:** negative renovation cost.
- **WAIT (holding):** negative maintenance cost.
- **WAIT (idle):** negative idle-capital penalty.
- **Invalid action:** negative penalty.
- **Terminal:** forced-sale penalty if the agent still owns the property at the end.

2.5 Episode Termination

An episode ends if one of the following conditions occurs:

1. The horizon $T = 120$ months is reached.
2. Capital falls below zero (bankruptcy).
3. During evaluation only, capital exceeds the success threshold.

3. Environment Implementation and Validation

We implemented the MDP as a custom `gymnasium.Env` subclass in `src/env.py`.

3.1 Random Policy Validation

Following the workshop guidelines, we validated the environment by running **100 episodes with a uniform random policy**. The results were:

- **Mean Return:** -3.53 ± 1.74.
- **Mean Length:** 95.3 steps (77 reached the horizon, 23 went bankrupt).
- **Action Frequencies:** approximately 25% each.

The fact that 23% of random agents go bankrupt confirms that the capital dynamics and bankruptcy termination are correctly implemented.

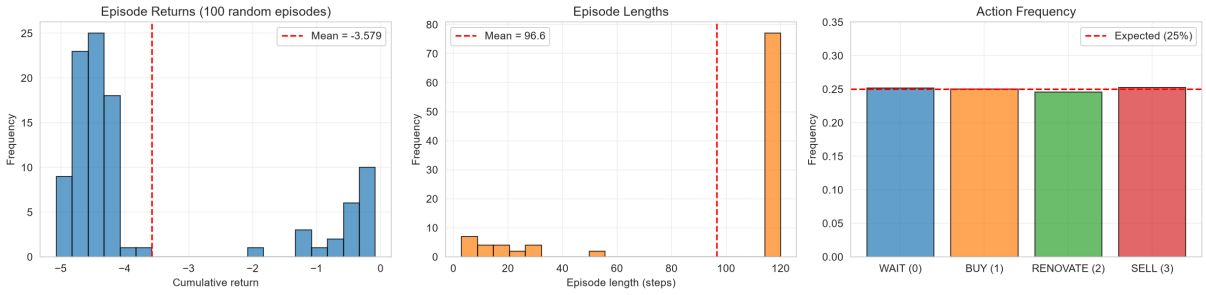


Figure 1: Random policy diagnostics: return distribution, episode length distribution, and action frequency over 100 random episodes.

4. Agent Algorithm (DQN)

We chose a **Deep Q-Network (DQN)** because our state space is continuous ($\mathbb{R}^8$) and our action space is discrete.

4.1 Architecture

- **Q-Network:** MLP with 8 inputs $\rightarrow$ 128 $\rightarrow$ 128 $\rightarrow$ 4 outputs (18,180 parameters).
- **Target Network:** updated every 1,000 steps.
- **Experience Replay:** buffer of 100,000 transitions, batch size 64.
- **Exploration:** ε-greedy with decay from 1.0 to 0.05.
- **Optimizer:** Adam with learning rate $= 10^{-3}$ and gradient clipping.

4.2 Training Dynamics

Training was completed in 2.6 minutes on CPU (1,000 episodes). The agent improved its training return from -3.5 to -0.51, and reached a maximum validation return of $+0.0143$.

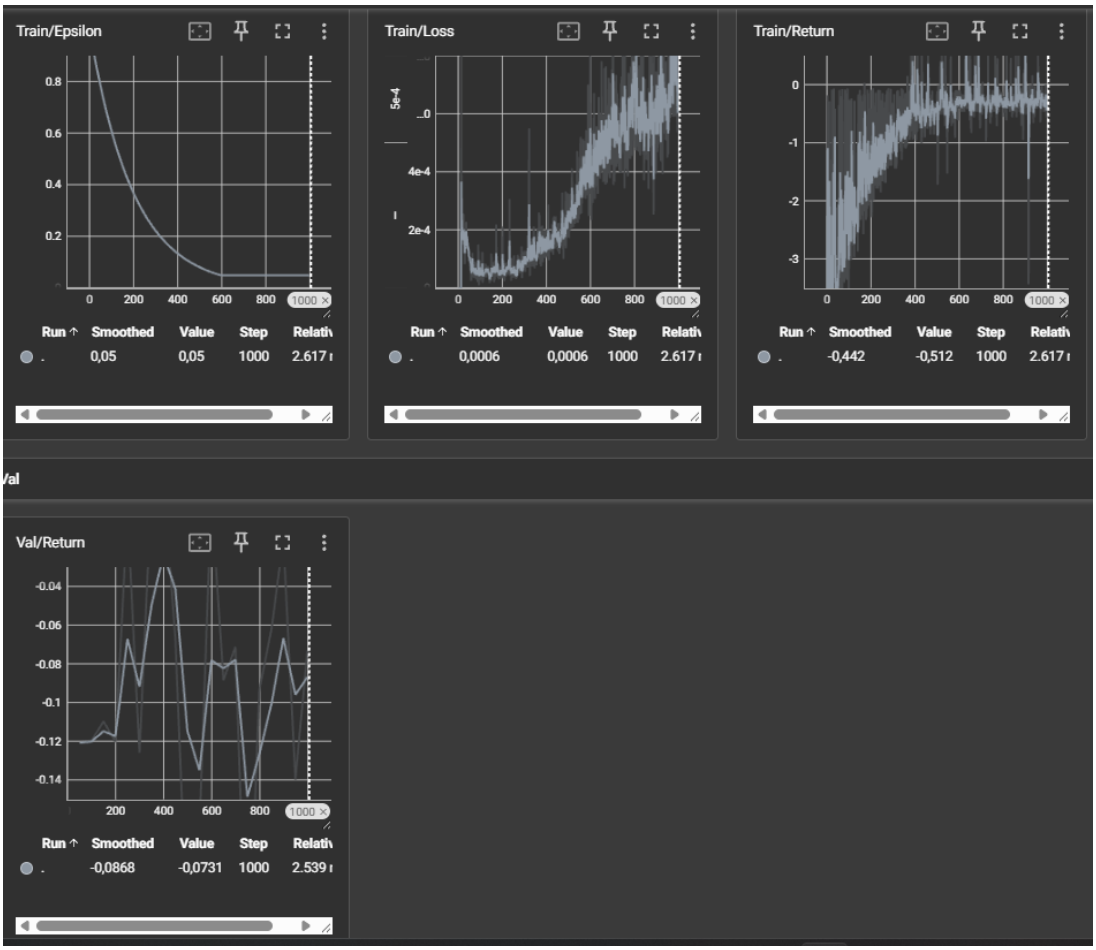


Figure 2: TensorBoard training diagnostics: epsilon decay, loss evolution, and training return over 1,000 episodes.

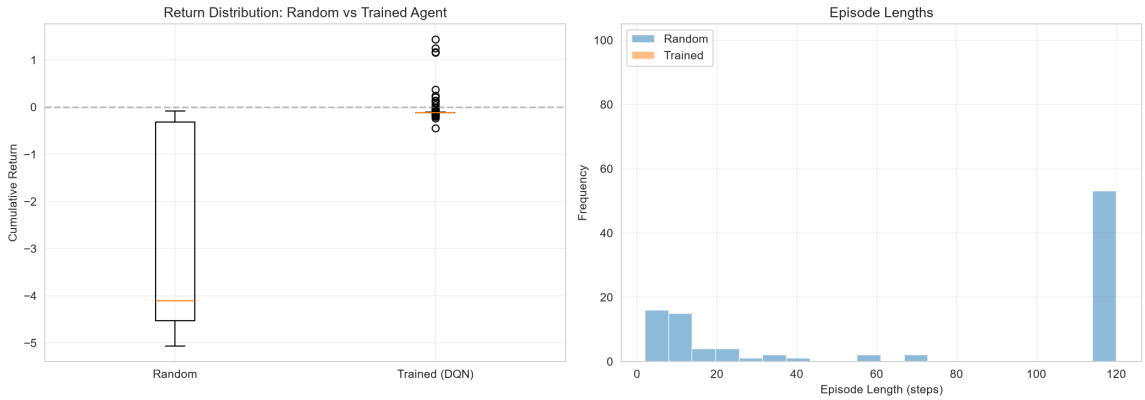


Figure 3: Comparison of the performance of the evaluated reinforcement learning agents.

5. Hyperparameter Search

We trained three additional configurations to address the policy degeneration (WAIT-only behavior):

1. **Aggressive:** increased idle-capital penalty ($0.001 \rightarrow 0.010$).
2. **Patient Investor:** increased discount factor ($\gamma = 0.99 \rightarrow 0.999$).
3. **High Learning Rate:** increased learning rate ($10^{-3} \rightarrow 5 \times 10^{-3}$).

5.1 Results Table

Table 1: Hyperparameter search results

Configuration	Mean Return	Bankruptcy Rate (%)	WAIT Frequency (%)
Baseline (Notebook 03)	−0.065	0.0	96.0
Aggressive	−0.259	32.0	91.1
Patient Investor	−0.316	29.0	93.7
High LR	−0.098	12.0	95.8

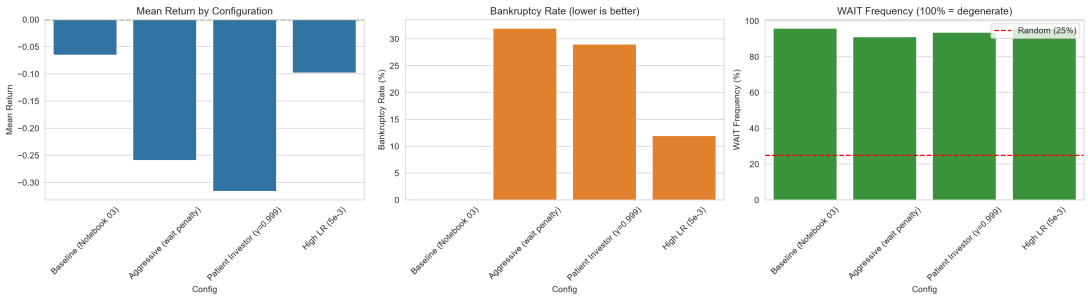


Figure 4: Results obtained during the hyperparameter search process.

5.2 Analysis

The baseline remained the most stable. Forcing action caused a massive increase in bankruptcies (32% and 29%). The reason is the **transaction cost ceiling**: with 3% buying, 5% selling, and 0.2% maintenance costs, the agent would need a monthly price appreciation greater than 8% to profit. Given our 1.4% monthly volatility, waiting is mathematically the optimal survival strategy.

6. Evaluation and Comparison

We evaluated the best DQN checkpoint against two baselines on **100 unseen test episodes** (seeds 50000+): Random Policy and Buy-and-Hold Heuristic.

6.1 Final Test Metrics

Table 2: Final evaluation metrics on 100 test episodes

Policy	Mean Return	Std Return	Max Return	Bankruptcy (%)	Mean Length
Random	−3.227	1.931	−0.080	32.0	86.15
Buy-and-Hold	−2.994	2.950	−0.002	51.0	76.03
DQN Agent	−0.027	**0.283**	+1.540	**0.0**	**120.00**

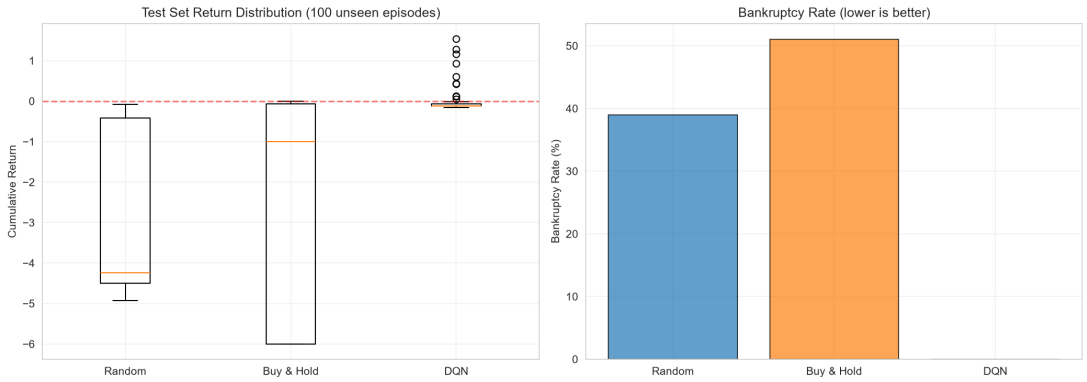


Figure 5: Final evaluation results of the selected reinforcement learning agent.

6.2 Comparison with Workshops 1 and 2

While W1/W2 required approximately 1,460 records for supervised learning, the DQN required 120,000 environment interactions. This confirms that RL is sample-inefficient compared to supervised learning. However, RL learned something W1/W2 could not: *when* to act, not just *what the price is*.

7. Trajectory Visualizations

The figure below shows three test episodes to illustrate the agent's behavior:

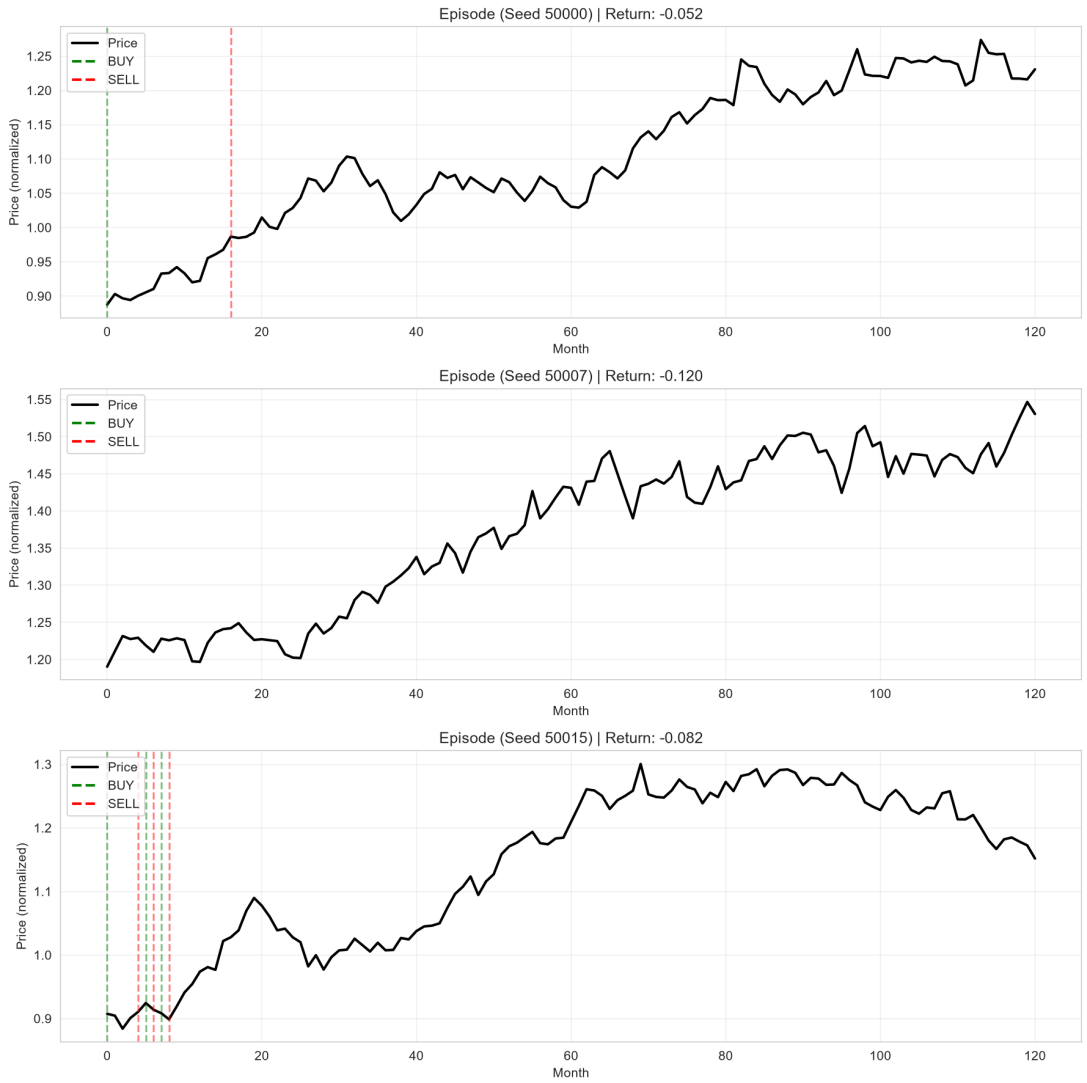


Figure 6: Agent trajectories illustrating the decisions made during the real estate investment process.

- **Episode 50000:** The agent bought low and sold high. The logic was correct, but transaction costs ate the profit.
- **Episode 50007:** The agent waited the entire episode, missing a price rally. This reflects the "degenerate policy" learned during training.
- **Episode 50015:** The agent attempted day-trading, but transaction costs resulted in a net loss.

8. Discussion

Did RL outperform supervised learning?

Not in profitability, but in *stability*. The DQN agent avoided bankruptcy completely (0%), while the traditional Buy-and-Hold strategy failed 51% of the time. The supervised models from W1/W2 predicted prices accurately, but could not make investment decisions.

What were the key challenges?

1. **Policy degeneration:** The agent learned to WAIT to avoid costly trades, driven by high transaction costs.
2. **Sample inefficiency:** RL required 120,000 interactions, versus approximately 1,460 records for supervised learning.
3. **Exploration vs. exploitation:** Maintaining a high epsilon early was crucial; without it, the agent would have converged to the WAIT-only policy faster.

How to incentivize active trading (Future Work):

1. Reduce transaction costs (e.g., 3% $\rightarrow$ 1%).
2. Introduce a rental income stream during the holding period.
3. Add reward shaping that penalizes holding cash while prices trend upwards.

9. References

1. Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.
2. Watkins, C. J. C. H., & Dayan, P. (1992). Q-Learning. *Machine Learning*, 8(3–4), 279–292.
3. Mnih, V., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533.
4. De Cock, D. (2011). Ames, Iowa: Alternative to the Boston Housing Data as an End of Semester Regression Project. *Journal of Statistics Education*, 19(3).
5. Towers, M., et al. (2024). Gymnasium: A Standard Interface for Reinforcement Learning Environments. *arXiv preprint*.